

IU Internationale Hochschule

Fallstudie

Im Studiengang
„Medieninformatik“

Programmierung von Webanwendungsoberflächen
IPWA01-01

Tutor
Alexander Christopher Bock

Quellcode-Repository:
<https://github.com/IUonas/IPWA01-01>

Eingereicht von
Jonas Dold
Matrikelnummer IU14118750
Jonas.Dold@iu-study.org

am
01.05.2026

Inhaltsverzeichnis

Titelblatt

Inhaltsverzeichnis

Quellcodeverzeichnis

1. Einleitung

2. Fallvorstellung

3. Konzeption und Umsetzung

3.1 Zielsetzung und Anforderungen

3.2 Technische Konzeption

3.3 Umsetzung der Anwendung

4. Systemarchitektur & Umsetzung

4.1 Gesamtaufbau der Webanwendung

4.2 Datenfluss & Datenverarbeitung

4.3 Sicherheitsaspekt

4.4 Benutzeroberfläche und Layout

4.5 Tabellenlogik und Dateninteraktion

4.5.1 Einsatz des Tabulator-Frameworks

4.5.2 Manuell Implementierte Tabellenstruktur

4.5.3 Vergleich der Ansätze und Sicherheitsaspekt

4.6 Visualisierung und Interaktionslogik

5. Schlussbetrachtung und Bewertung der Umsetzung

Literaturverzeichnis

Quellcodeverzeichnis

Listing 1: Laden der JSON-Daten mittels fetch()

Listing 2: Responsive Layoutstruktur mit Bootstrap

Listing 3: Eigenes CSS-Grid für die Technologie-Karten

Listing 4: Initialisierung der Tabulator-Tabelle

Listing 5: Definition der Tabellenspalten mit Sortier- und Filterfunktion

Listing 6: Implementierung der Such- und Filterfunktion

Listing 7: Initialisierung und Neuerstellung des Diagramms mit Chart.js

1. Einleitung

Die Reduktion von CO₂-Emissionen stellt eine der zentralen Herausforderungen im Kontext des globalen Klimawandels dar. Treibhausgasemissionen gelten als maßgeblicher Faktor für die Erderwärmung und erfordern sowohl auf politischer als auch auf wirtschaftlicher Ebene gezielte Maßnahmen (Umweltbundesamt 2023).

Um fundierte Entscheidungen im Bereich des Umwelt- und Klimaschutzes treffen zu können, ist eine transparente und verständliche Aufbereitung relevanter Daten von entscheidender Bedeutung. In der Praxis stehen jedoch häufig große und komplexe Datenmengen zur Verfügung, die ohne geeignete Werkzeuge nur schwer analysierbar sind. Insbesondere Emissionsdaten von Unternehmen oder staatlichen Institutionen werden zwar erhoben, sind jedoch oft nicht in einer Form aufbereitet, die eine intuitive Nutzung ermöglicht.

Digitale Anwendungen bieten hier einen entscheidenden Mehrwert. Durch die Kombination aus strukturierter Datendarstellung und interaktiven Analysefunktionen können komplexe Zusammenhänge verständlich visualisiert werden. Die Aufbereitung und Visualisierung von Umweltdaten spielt dabei eine zentrale Rolle, um Transparenz zu schaffen und datenbasierte Entscheidungen zu unterstützen (European Environment Agency 2022).

Die Aufgabenstellung der Fallstudie ist dabei aus der Perspektive einer großen Non-Profit-Organisation zu betrachten, die sich mit dem Klimawandel beschäftigt. Für eine solche Organisation steht nicht die kommerzielle Nutzung der Daten im Vordergrund, sondern die öffentliche Bereitstellung verständlicher Informationen. Die Anwendung muss daher niedrigschwellig zugänglich sein und auch von Personen genutzt werden können, die keine technischen oder datenanalytischen Vorkenntnisse besitzen. Daraus ergibt sich die Notwendigkeit einer klaren Benutzerführung, einer verständlichen Darstellung der Emissionsdaten sowie interaktiver Funktionen, mit denen relevante Informationen gezielt gefiltert und verglichen werden können.

Vor diesem Hintergrund wird im Rahmen der vorliegenden Fallstudie eine webbasierte Anwendung entwickelt, die CO₂-Emissionsdaten darstellt und analysierbar macht. Ziel ist es, eine benutzerfreundliche Oberfläche bereitzustellen, die eine tabellarische und grafische Aufbereitung der Daten ermöglicht. Dabei stehen insbesondere interaktive Funktionen wie Filter- und Sortiermechanismen im Fokus, um eine flexible Analyse zu unterstützen.

Neben der funktionalen Umsetzung liegt ein weiterer Schwerpunkt auf der technischen Realisierung der Anwendung. Es wird untersucht, wie moderne Webtechnologien eingesetzt werden können, um strukturierte Daten effizient zu verarbeiten und gleichzeitig eine intuitive Benutzeroberfläche bereitzustellen.

2. Fallvorstellung

Im Rahmen dieser Fallstudie wurde eine webbasierte Anwendung mit dem Namen „ATMOS Track“ entwickelt, die der strukturierten Darstellung und Analyse von CO₂-Emissionsdaten dient. Ziel der Anwendung ist es, komplexe Datensätze zu vereinfachen und Nutzenden eine intuitive Möglichkeit zu bieten, Emissionen nach verschiedenen Kriterien zu untersuchen.

Die Anwendung basiert auf einer klassischen Webarchitektur und besteht aus mehreren miteinander verknüpften Seiten. Eine zentrale Startseite bietet einen thematischen Einstieg in die Anwendung und ermöglicht die Navigation zu den verschiedenen Funktionsbereichen.

Ein wesentlicher Bestandteil der Anwendung ist die tabellarische Darstellung der Emissionsdaten. Diese erfolgt auf einer eigenen Seite, auf der die Daten interaktiv gefiltert, sortiert und analysiert werden können. Die Tabelle stellt Informationen zu Unternehmen, Ländern, Branchen, Jahren sowie Emissionswerten und deren Veränderungen dar.

Ergänzend zur Tabellenansicht wurde eine visuelle Komponente integriert, die die Daten in Form von Diagrammen aufbereitet. Abhängig vom Filterstatus der Tabelle werden unterschiedliche Diagrammtypen verwendet, um entweder eine aggregierte Übersicht oder detaillierte Einzelwerte darzustellen. Dies ermöglicht eine bessere Erkennung von Mustern und Zusammenhängen innerhalb der Daten.

Neben der Framework-basierten Tabellenlösung wurde zusätzlich eine alternative, manuell entwickelte Tabellenvariante umgesetzt. Diese dient dazu, grundlegende Mechanismen der Datenverarbeitung und Darstellung ohne den Einsatz externer Bibliotheken zu verstehen und zu demonstrieren.

Die zugrunde liegenden Daten werden in Form einer JSON-Datei bereitgestellt und enthalten mehrere Datensätze zu unterschiedlichen Unternehmen, Ländern und Branchen. Diese Daten bilden die Grundlage für alle dargestellten Funktionen und werden dynamisch in die Anwendung geladen und verarbeitet.

3. Konzeption und Umsetzung

Die Entwicklung der Anwendung erfolgte auf Basis einer strukturierten Konzeption, die sowohl funktionale Anforderungen als auch technische Umsetzungsaspekte berücksichtigt. Ziel war es, eine interaktive Webanwendung zu realisieren, die eine übersichtliche Darstellung und Analyse von CO₂-Emissionsdaten ermöglicht.

3.1 Zielsetzung und Anforderungen

Im Mittelpunkt der Anwendung steht die verständliche Aufbereitung komplexer Emissionsdaten. Daraus ergeben sich mehrere zentrale Anforderungen an die Umsetzung der Webanwendung, die sich an den Vorgaben der Aufgabenstellung orientieren.

Die Anwendung muss dabei insbesondere folgende Kriterien erfüllen:

- Die Webseite besitzt einen Titel sowie ein visuelles Logo zur eindeutigen Identifikation
- Die Webseite verfügt über einen Header mit globaler Navigation, einen strukturierten Content-Bereich sowie einen Footer mit rechtlichen Hinweisen
- Es wird ein lokales Menü integriert, das je nach Schriftkultur (links-nach-rechts bzw. rechts-nach-links) unterschiedlich positioniert werden kann
- Die Anwendung ist responsiv gestaltet, sodass sie auf Desktop-, Tablet- und mobilen Endgeräten nutzbar ist
- Es wird eine Tabelle mit CO₂-Emissionsdaten bereitgestellt, die nach Land und Unternehmen sortiert und gefiltert werden kann
- Eingabefelder werden abgesichert, sodass kein injizierter Code ausgeführt werden kann

Neben diesen funktionalen Anforderungen spielt auch die Benutzerfreundlichkeit eine zentrale Rolle. Die Anwendung soll intuitiv bedienbar sein und eine klare visuelle Struktur aufweisen, um eine effiziente Nutzung zu ermöglichen.

3.2 Technische Konzeption

Auf Grundlage der definierten Anforderungen wurde eine technische Architektur entwickelt, die auf einer Kombination aus HTML, CSS und JavaScript basiert. Ergänzend wurden gezielt Frameworks und Bibliotheken eingesetzt, um wiederkehrende Aufgaben effizient umzusetzen.

Für die Gestaltung der Benutzeroberfläche wurde das CSS-Framework Bootstrap gewählt (Bootstrap o. J.). Dieses ermöglicht eine schnelle Umsetzung eines responsiven Layouts und stellt zahlreiche vordefinierte Komponenten zur Verfügung. Für die tabellarische Darstellung der Daten wurde das Tabulator-Framework eingesetzt, da es umfangreiche Funktionen wie Filterung, Sortierung und Pagination integriert bereitstellt (Tabulator o. J.). Die visuelle Aufbereitung der Daten erfolgt mithilfe der Bibliothek Chart.js, die eine flexible Erstellung dynamischer Diagramme ermöglicht (Chart.js o. J.). Die Auswahl dieser Technologien erfolgte bewusst, um den Entwicklungsaufwand zu reduzieren und gleichzeitig eine leistungsfähige und skalierbare Lösung zu realisieren. Gleichzeitig wurde darauf geachtet, die Funktionsweise der eingesetzten Tools nachvollziehen zu können.

Zur Gestaltung der Benutzeroberfläche wurden gängige Webdesign-Prinzipien sowie visuelle Orientierungen an bestehenden Webanwendungen herangezogen. Ziel war es eine klare visuelle Hierarchie sowie eine intuitive Benutzerführung zu schaffen.

3.3 Umsetzung der Anwendung

Zur eindeutigen Identifikation der Anwendung wurde ein Titel sowie ein visuelles Logo integriert, die auf allen Seiten sichtbar sind und eine konsistente Wiedererkennung gewährleisten. Bei der konzeptionellen Entwicklung des Logos wurde ein Vorschlag eines KI-gestützten Tools aufgegriffen (OpenAI, 2026). Die anschließende grafische Umsetzung erfolgte mithilfe von Adobe Photoshop.

Die grundlegende Seitenstruktur wurde durch die Implementierung eines Headers, eines Content-Bereichs sowie eines Footers realisiert. Der Header enthält eine globale Navigationsleiste, über die alle zentralen Seiten der Anwendung erreichbar sind. Der Footer ergänzt die Anwendung um rechtliche Inhalte wie Impressum, Datenschutz und Kontakt und erfüllt damit die Anforderungen an eine vollständige Webpräsenz.

Ergänzend zur globalen Navigation wurde ein lokales Dropdown-Menü mit Ankerlinks implementiert, das kontextabhängige Inhalte innerhalb einer Seite zugänglich macht. Dabei wurde berücksichtigt, dass sich die Positionierung des Menüs an unterschiedliche Schriftkulturen anpassen lässt, wodurch eine flexible Nutzung in verschiedenen Kontexten unterstützt wird.

Die Umsetzung eines responsiven Designs erfolgte mithilfe des Bootstrap-Frameworks. Durch das Grid-System sowie vordefinierte Komponenten passt sich die Darstellung automatisch an unterschiedliche Bildschirmgrößen an. Inhalte werden auf größeren Bildschirmen nebeneinander und auf mobilen Geräten untereinander dargestellt. Zusätzlich wird die Navigationsleiste auf kleineren Geräten in ein kompaktes Menü überführt, um die Bedienbarkeit zu verbessern.

Die zentrale funktionale Anforderung – die Darstellung und Analyse von CO₂-Emissionsdaten – wurde durch eine interaktive Tabelle umgesetzt. Hierbei kommt das Tabulator-Framework zum Einsatz, das Sortier- und Filterfunktionen bereitstellt. Nutzende können die Daten nach Kriterien wie Land oder Unternehmen durchsuchen und anpassen, wodurch eine flexible Analyse ermöglicht wird.

Ergänzend dazu wurde eine manuelle Tabellenvariante ohne Framework entwickelt. Diese macht grundlegende Mechanismen wie das Rendern von Daten, die Sortierung sowie die Filterung nachvollziehbar und dient zugleich als Vergleichsbasis zur Framework-Lösung.

Ein weiterer wichtiger Aspekt betrifft die Absicherung von Eingabefeldern. Benutzereingaben, beispielsweise in Suchfeldern, werden ausschließlich zur Filterung von Daten verwendet und nicht als ausführbarer Code interpretiert. In der manuellen Implementierung erfolgt zudem eine Vorverarbeitung der Eingaben, um diese zu normalisieren. Darüber hinaus bietet das Tabulator-Framework integrierte Mechanismen zur sicheren Verarbeitung von Eingaben, wodurch das Risiko von Code-Injektionen reduziert wird.

Insgesamt zeigt die Umsetzung, dass alle definierten Anforderungen systematisch berücksichtigt und in eine funktionale sowie benutzerfreundliche Anwendung überführt wurden. Die Kombination aus strukturierter Seitenarchitektur, interaktiver Datenverarbeitung und responsivem Design bildet die Grundlage für die im folgenden Kapitel detailliert analysierte Systemarchitektur.

4. Systemarchitektur & Umsetzung

4.1 Gesamtaufbau der Webanwendung

Die entwickelte Anwendung folgt einer mehrseitigen Webarchitektur. Sie besteht aus mehreren HTML-Dokumenten, die jeweils eine eigene Funktion innerhalb des Gesamtsystems übernehmen. Dazu zählen unter anderem die Startseite, die Tabellenansicht, die manuelle Tabellenvariante sowie die Seite zur Darstellung der verwendeten Technologien. Die Seiten sind über eine zentrale Navigation miteinander verbunden und ermöglichen dadurch eine konsistente Benutzerführung.

Ein wesentliches Merkmal der Architektur ist die Trennung von Struktur, Gestaltung und Logik. Die HTML-Dateien bilden die semantische Grundstruktur, während das visuelle Erscheinungsbild über eine zentrale CSS-Datei gesteuert wird. Die funktionale Logik wird in JavaScript-Dateien umgesetzt, die je nach Seite Aufgaben wie Datenverarbeitung, Tabelleninteraktion oder Visualisierung übernehmen.

Die Navigation zwischen den einzelnen Seiten erfolgt über eine global eingebundene Navigationsleiste, die auf allen Seiten identisch implementiert ist. Dadurch wird eine konsistente Benutzerführung gewährleistet. Ergänzend dazu können innerhalb einzelner Seiten lokale Navigationsstrukturen eingesetzt werden, um spezifische Inhalte gezielt anzusteuern.

Ein weiterer zentraler Bestandteil ist die Einbindung externer Bibliotheken und Frameworks. Diese werden über Content Delivery Networks (CDN) geladen und stehen somit direkt innerhalb der jeweiligen HTML-Seiten zur Verfügung. Dieses Vorgehen ermöglicht eine einfache Integration sowie eine Reduzierung des lokalen Verwaltungsaufwands.

Die einzelnen Komponenten der Anwendung sind über klar definierte Schnittstellen miteinander verbunden. So werden beispielsweise Daten aus einer zentralen JSON-Datei geladen und anschließend sowohl an die Tabellenlogik als auch an die Visualisierung übergeben. Dadurch entsteht ein durchgängiger Datenfluss innerhalb der Anwendung.

Insgesamt zeigt der Aufbau der Webanwendung eine modulare und klar strukturierte Architektur, die sowohl eine einfache Erweiterbarkeit als auch eine nachvollziehbare Trennung der einzelnen Systembestandteile ermöglicht. Diese Struktur bildet die Grundlage für die nachfolgende detaillierte Analyse der Datenverarbeitung und Interaktionslogik.

4.2 Datenfluss & Datenverarbeitung

Ein zentraler Bestandteil der Anwendung ist die strukturierte Verarbeitung der Emissionsdaten. Diese bildet die Grundlage für die in Kapitel 3 definierten Anforderungen, insbesondere für die interaktive Darstellung sowie die Analysefunktionen der Tabelle. Die Daten liegen in Form einer JSON-Datei vor und werden beim Laden der Seite asynchron über die `fetch`-Funktion abgerufen. Nach erfolgreichem Laden werden die Daten in ein JavaScript-Objekt überführt und an eine zentrale Initialisierungsfunktion übergeben:

```
fetch("data/data.json")
  .then(response => response.json())
  .then(data => {
    initApp(data);
  });
```

Listing 1: Laden der JSON-Daten mittels `fetch()`

Innerhalb der Funktion `initApp()` werden die geladenen Daten für die weitere Verarbeitung vorbereitet und an die jeweiligen Komponenten der Anwendung übergeben. Dazu zählen insbesondere die tabellarische Darstellung sowie die Visualisierung der Daten. Die vollständige Implementierung ist im Anhang dargestellt (vgl. Anhang A).

Die Darstellung und Aktualisierung der Inhalte erfolgt durch die dynamische Manipulation des Document Object Model (DOM). Dabei wird die HTML-Struktur der Seite zur Laufzeit angepasst, indem neue Elemente erzeugt oder bestehende Inhalte verändert werden. Auf diese Weise können Daten interaktiv verarbeitet und angezeigt werden, ohne dass ein vollständiges Neuladen der Seite erforderlich ist.

Die konkrete Umsetzung der Tabellenlogik, einschließlich Sortier-, Filter- und Interaktionsmechanismen, wird in Abschnitt 4.5 detailliert erläutert.

4.3 Sicherheitsaspekt

Auch wenn es sich bei der entwickelten Anwendung um eine vergleichsweise einfache Demonstrationswebseite handelt, spielen Sicherheitsaspekte eine wichtige Rolle. Webanwendungen sind grundsätzlich potenziellen Risiken ausgesetzt, weshalb grundlegende Schutzmaßnahmen bereits bei der Konzeption berücksichtigt werden sollten.

Ein zentraler Aspekt betrifft den Umgang mit Benutzereingaben. Eingaben können potenziell missbraucht werden, weshalb sie grundsätzlich validiert und verarbeitet werden sollten, um Sicherheitsrisiken zu minimieren.

Darüber hinaus ist die Herkunft und Übertragung von Daten zu berücksichtigen. In realen Anwendungen sollte sichergestellt werden, dass Daten ausschließlich über gesicherte Verbindungen übertragen werden und keine unbefugte Manipulation möglich ist.

Ein weiterer relevanter Punkt ist der Einsatz externer Bibliotheken und Frameworks. Diese stellen zwar eine erhebliche Erleichterung in der Entwicklung dar, können jedoch auch potenzielle Sicherheitsrisiken mit sich bringen. Daher sollten ausschließlich vertrauenswürdige Quellen verwendet und eingesetzte Komponenten regelmäßig aktualisiert werden.

Die konkrete Umsetzung sicherheitsrelevanter Maßnahmen innerhalb der Anwendung, insbesondere im Kontext der Datenverarbeitung und Benutzereingaben, wird in Abschnitt 4.5.3 näher betrachtet.

4.4 Benutzeroberfläche und Layout

Die Gestaltung der Benutzeroberfläche erfolgt durch eine Kombination aus dem Bootstrap-Framework sowie ergänzenden eigenen CSS-Definitionen. Dabei werden zwei unterschiedliche Ansätze zur Layoutstrukturierung eingesetzt: das Bootstrap-basierte Spaltensystem sowie ein eigenständig definiertes Grid-Layout.

Ein zentraler Bestandteil der Layoutgestaltung ist das von Bootstrap bereitgestellte Spaltensystem, welches auf sogenannten „Rows“ und „Columns“ basiert. Innerhalb einer `row` werden einzelne Inhalte über Klassen wie `col-lg-4` oder `col-lg-8` in Spalten unterteilt. Dieses Prinzip wird insbesondere auf der Tabellen-Seite genutzt, um Diagramm und Tabelle nebeneinander darzustellen:

```
<div class="row g-4 align-items-center">
  <div class="col-lg-4 col-12">
    <!-- Chart -->
  </div>
  <div class="col-lg-8 col-12">
    <!-- Tabelle -->
  </div>
</div>
```

Listing 2: Responsive Layoutstruktur mit Bootstrap

Hierbei wird deutlich, dass die verfügbare Breite des Containers in ein Verhältnis von 4 zu 8 Spalten aufgeteilt wird. Auf großen Bildschirmen entstehen so zwei nebeneinanderliegende Bereiche, während durch die zusätzliche Klasse `col-12` auf kleineren Geräten eine automatische Umstellung auf eine untereinanderliegende Darstellung erfolgt. Dieses Verhalten ist ein wesentliches Merkmal des responsiven Designs von Bootstrap.

Bootstrap verwendet hierfür ein sogenanntes Breakpoint-System, das unterschiedliche Bildschirmgrößen berücksichtigt. Klassen wie `col-lg-*` beziehen sich dabei auf definierte Bildschirmbreiten („large devices“), die typischerweise ab einer Breite von 992 Pixeln greifen. Für kleinere Geräte stehen weitere Abstufungen wie `col-sm` (kleine Geräte), `col-md` (mittlere Geräte) oder `col-xl` (sehr große Bildschirme) zur Verfügung.

Durch diese abgestuften Klassen kann das Layout gezielt an verschiedene Endgeräte angepasst werden, ohne dass separate CSS-Regeln implementiert werden müssen. Inhalte können so je nach Bildschirmgröße unterschiedlich angeordnet werden, was eine zentrale Grundlage für die Umsetzung responsiver Webanwendungen darstellt.

Neben diesem spaltenbasierten Ansatz wird in der Anwendung zusätzlich ein eigenes Grid-System über CSS verwendet. Ein Beispiel hierfür ist die Technologie-Seite, auf der mehrere Karten in einem gleichmäßigen Raster angeordnet sind. Dieses Layout wird nicht über Bootstrap, sondern über eine eigene CSS-Klasse umgesetzt:

```
.tech-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

Listing 3: Eigenes CSS-Grid für die Technologie-Karten

Durch die Eigenschaft `display: grid` wird das Element `.tech-grid` zu einem Grid-Container. Die darin enthaltenen Karten werden dadurch automatisch als Grid-Elemente behandelt und innerhalb eines Rasters angeordnet. Mit `grid-template-columns: repeat(3, 1fr)` wird festgelegt, dass das Raster aus drei gleich breiten Spalten besteht. Die Angabe `1fr` steht dabei für einen Anteil des verfügbaren freien Platzes. Da drei Spalten mit jeweils `1fr` definiert werden, wird die verfügbare Breite gleichmäßig auf alle drei Spalten verteilt.

Die Eigenschaft `gap: 1rem` definiert den Abstand zwischen den einzelnen Rasterelementen. Dadurch müssen keine zusätzlichen Außenabstände an den Karten selbst gesetzt werden, was die Struktur übersichtlich hält und ein gleichmäßiges Layout erzeugt.

Im Vergleich zum Bootstrap-Spalstensystem bietet CSS Grid eine direktere Kontrolle über die Anordnung der Elemente. Während Bootstrap mit vordefinierten Klassen wie `row`, `col-lg-4` oder `col-12` arbeitet, wird das Layout bei CSS Grid vollständig über eigene CSS-Regeln gesteuert. Dadurch kann die Struktur unabhängig von einem Framework angepasst werden.

Ein weiterer Vorteil besteht darin, dass CSS Grid besonders gut für zweidimensionale Layouts geeignet ist. Elemente können nicht nur in Spalten, sondern gleichzeitig auch in Zeilen organisiert werden. Dies ist insbesondere bei Kartenlayouts hilfreich, da mehrere Inhalte in einem gleichmäßigen Raster dargestellt werden können. In der Anwendung wird dieser Ansatz genutzt, um die Technologie-Karten optisch geordnet und leicht erfassbar darzustellen.

Die Kombination aus Bootstrap-Grid und eigenem CSS Grid ermöglicht somit eine flexible Gestaltung der Benutzeroberfläche. Bootstrap wird vor allem für die übergeordnete Seitenstruktur und responsive Spaltenaufteilung eingesetzt, während CSS Grid gezielt für spezifische Inhaltsbereiche verwendet wird, in denen eine individuelle und gleichmäßige Rasterdarstellung sinnvoll ist.

Neben der grundlegenden Strukturierung werden auch vordefinierte Komponenten genutzt, um eine konsistente Benutzeroberfläche zu gewährleisten. Dazu zählen insbesondere Navigationsleisten, Karten (Cards) sowie Buttons. Die Navigationsleiste ist auf allen Seiten identisch aufgebaut und stellt die zentrale Steuerung der Anwendung dar. Durch die wiederholte Verwendung derselben Struktur wird eine hohe Wiedererkennbarkeit erreicht, was die Benutzerführung verbessert.

Die Umsetzung der Navigationsleiste erfolgt mithilfe der Bootstrap-Komponente „Navbar“, die eine Kombination aus strukturellem HTML, vordefinierten CSS-Klassen und integrierter JavaScript-Logik darstellt. Dabei wird die Navigation nicht nur visuell gestaltet, sondern auch funktional erweitert. Insbesondere das responsive Verhalten basiert auf dem Konzept der sogenannten Breakpoints: Während die Navigation auf größeren Bildschirmen vollständig horizontal dargestellt wird, erfolgt auf kleineren Geräten eine Transformation in ein kompaktes Menü. Diese Umwandlung wird durch die Klasse `navbar-expand-lg` gesteuert, welche definiert, ab welcher Bildschirmgröße die horizontale Darstellung aktiv ist. Unterhalb dieses Breakpoints wird die Navigation automatisch reduziert und über ein ausklappbares Menü zugänglich gemacht. Die konkrete Implementierung der Navigationsstruktur ist im Anhang dargestellt (vgl. Anhang B, Listing B1).

Die technische Umsetzung dieser Interaktion erfolgt über deklarative Attribute wie `data-bs-toggle="collapse"` und `data-bs-target`. Diese verknüpfen das sichtbare Steuerelement (den sogenannten „Hamburger-Button“) mit dem Navigationscontainer, sodass dieser ein- und ausgeblendet werden kann. Entscheidender Vorteil dieses Ansatzes ist, dass komplexe

Interaktionen ohne eigene JavaScript-Implementierung realisiert werden können. Bootstrap kapselt die zugrunde liegende Logik und stellt sie in abstrahierter Form bereit, wodurch Entwicklungsaufwand reduziert und gleichzeitig standardisierte, getestete Funktionalität genutzt wird.

Darüber hinaus basiert die Navigationsstruktur auf semantischen HTML-Elementen wie `<nav>`, `<ul>` und `<li>`, die eine klare hierarchische Gliederung der Inhalte ermöglichen. Diese semantische Struktur verbessert nicht nur die Lesbarkeit des Codes, sondern unterstützt auch die Zugänglichkeit der Anwendung, beispielsweise für Screenreader. Insgesamt zeigt sich, dass die Navigation nicht nur ein gestalterisches Element darstellt, sondern durch das Zusammenspiel von Struktur, Stil und Verhalten eine zentrale Rolle für die Benutzerführung und Interaktion innerhalb der Anwendung einnimmt.

Ergänzend zum Bootstrap-Framework wird eine eigene CSS-Datei verwendet, um das Erscheinungsbild weiter anzupassen. Hierdurch können standardisierte Komponenten individuell gestaltet werden, beispielsweise durch Anpassung von Farben, Abständen oder Schatteneffekten. Dies ermöglicht eine Abgrenzung von Standarddesigns und trägt zur visuellen Identität der Anwendung bei.

Ein weiterer wichtiger Aspekt ist die Trennung von Struktur und Design. Während Bootstrap primär für die Layoutstruktur verantwortlich ist, werden spezifische Designanpassungen über eigene CSS-Klassen realisiert. Diese klare Aufgabenteilung erhöht die Wartbarkeit des Codes und erleichtert spätere Erweiterungen.

Zusammenfassend lässt sich festhalten, dass Bootstrap in der vorliegenden Anwendung nicht nur zur schnellen Umsetzung eines Layouts dient, sondern gezielt eingesetzt wird, um eine responsive, konsistente und benutzerfreundliche Oberfläche zu realisieren. Die Kombination aus Framework-Komponenten und individuellen Anpassungen ermöglicht dabei eine flexible und gleichzeitig strukturierte Gestaltung der Benutzeroberfläche.

4.5 Tabellenlogik und Dateninteraktion

Die tabellarische Darstellung der Emissionsdaten stellt die zentrale Funktion der Anwendung dar und erfüllt damit eine der wichtigsten Anforderungen der Fallstudie, nämlich die Möglichkeit zur interaktiven Analyse von CO₂-Emissionsdaten durch Sortierung und Filterung.

4.5.1 Einsatz des Tabulator-Frameworks

Die Implementierung der Tabelle erfolgt mithilfe des Tabulator-Frameworks, das eine leistungsfähige und flexible Darstellung strukturierter Daten ermöglicht. Die Initialisierung der Tabelle erfolgt über die Übergabe der Daten sowie einer Konfigurationsstruktur:

```
const table = new Tabulator("#emissionsTable", {
  data: emissionsData,
  layout: "fitColumns",
  pagination: true,
  paginationSize: 15,
});
```

Listing 4: Initialisierung der Tabulator-Tabelle

Die Definition der Spalten erfolgt über eine strukturierte Konfiguration, bei der jedem Feld ein entsprechendes Datenattribut zugewiesen wird:

```

columns: [
  { title: "Unternehmen", field: "company", sorter: "string",
headerFilter: "input" },
  { title: "Land", field: "country", sorter: "string", headerFilter:
"input" }
]

```

Listing 5: Definition der Tabellenspalten mit Sortier- und Filterfunktion

Durch die Verwendung von `sorter` und `headerFilter` werden zentrale Anforderungen direkt umgesetzt. Nutzende können die Daten sowohl nach Land als auch nach Unternehmen sortieren und filtern, wodurch eine flexible und gezielte Analyse ermöglicht wird.

Ein weiterer Vorteil liegt in der integrierten Ereignissteuerung des Frameworks. Änderungen an den sichtbaren Daten, beispielsweise durch Filterung, können abgefangen und für weitere Verarbeitungsschritte genutzt werden. Dies bildet die Grundlage für die Verknüpfung mit der Visualisierung.

Zusätzlich wird eine visuelle Hervorhebung von Emissionsveränderungen umgesetzt, bei der positive und negative Werte farblich unterschiedlich dargestellt werden. Dieses Prinzip unterstützt die schnelle visuelle Erfassung von Entwicklungen und entspricht gängigen Ansätzen der Datenvisualisierung. Die konkrete Implementierung dieser Logik ist im Anhang dargestellt (vgl. Anhang B, Listing B2).

Darüber hinaus unterstützt das Tabulator-Framework Funktionen wie Pagination, bei der große Datenmengen auf mehrere Seiten aufgeteilt werden, um die Übersichtlichkeit und Performance der Darstellung zu verbessern.

4.5.2 Manuell Implementierte Tabellenstruktur

Ergänzend zur Framework-basierten Lösung wurde eine manuelle Tabellenimplementierung umgesetzt. Ziel dieser Variante ist es, die grundlegenden Mechanismen der Datenverarbeitung und Darstellung ohne den Einsatz eines spezialisierten Tabellen-Frameworks nachvollziehbar zu machen. Dadurch wird sichtbar, welche Arbeitsschritte bei Frameworks wie Tabulator intern abstrahiert werden.

Die Daten werden zunächst aus der JSON-Datei geladen und in einem eigenen Array gespeichert. Dieses Array dient als Arbeitskopie für Sortier- und Filteroperationen, sodass die ursprünglichen Datensätze unverändert erhalten bleiben. Für die Ausgabe der Daten wird der Tabellenkörper über die ID `tableBody` angesprochen und bei jeder Aktualisierung neu aufgebaut.

Im Unterschied zu einer direkten Ausgabe über `innerHTML` werden die Tabellenzeilen und Tabellenzellen manuell über DOM-Methoden erzeugt. Dazu wird für jeden Datensatz zunächst ein neues `tr`-Element erstellt. Anschließend werden für die einzelnen Datenfelder jeweils `td`-Elemente erzeugt. Die eigentlichen Werte werden über die Eigenschaft `textContent` eingefügt (vgl. Anhang, Listing A2)

Diese Vorgehensweise hat neben der besseren Nachvollziehbarkeit auch sicherheitsrelevante Vorteile. Inhalte aus den Datensätzen werden nicht als HTML interpretiert, sondern ausschließlich als Text in die Tabellenzellen eingefügt. Dadurch wird verhindert, dass potenziell manipulierte Inhalte aus der Datenquelle als ausführbarer HTML- oder JavaScript-Code verarbeitet werden. Die manuelle Tabellenvariante erfüllt damit nicht nur die funktionale Aufgabe der Datendarstellung, sondern berücksichtigt zugleich die

Anforderung, Eingaben und Daten so zu behandeln, dass kein injizierter Code ausgeführt wird.

Die Filterung der Daten wird über ein Eingabefeld realisiert. Bei jeder Änderung des Suchfelds wird der eingegebene Text zunächst normalisiert, indem Leerzeichen am Anfang und Ende entfernt und alle Zeichen in Kleinbuchstaben umgewandelt werden. Anschließend wird geprüft, ob der Suchbegriff in den Feldern Unternehmen, Land oder Branche enthalten ist:

```
searchInput.addEventListener("input", function() {
    const filterText = searchInput.value.toLowerCase().trim();

    currentData = emissionsData.filter(entry => {
        return entry.company.toLowerCase().includes(filterText) ||
            entry.country.toLowerCase().includes(filterText);
    });

    renderTable(currentData);
});
```

Listing 6: Implementierung der Such- und Filterfunktion

Auch die Sortierung erfolgt manuell über Vergleichsfunktionen innerhalb des Datenarrays. Dabei wird das aktuell angezeigte Array nach einem bestimmten Feld, beispielsweise Unternehmen, Land oder Branche, alphabetisch sortiert und anschließend erneut gerendert. Die Darstellung der Tabelle ist somit direkt an den jeweils aktuellen Zustand der Daten gebunden.

Insgesamt zeigt die manuelle Implementierung, wie Tabelleninhalte durch JavaScript dynamisch erzeugt, gefiltert und sortiert werden können. Gleichzeitig verdeutlicht sie den höheren Implementierungsaufwand gegenüber einer Framework-Lösung, da Funktionen wie Rendering, Sortierung, Filterung und sichere Textausgabe eigenständig umgesetzt werden müssen.

4.5.3 Vergleich der Ansätze und Sicherheitsaspekte

Der Vergleich beider Ansätze zeigt deutliche Unterschiede im Implementierungsaufwand. Während Tabulator komplexe Funktionen wie Sortierung, Filterung und Pagination bereits integriert bereitstellt, erfordert die manuelle Umsetzung eine vollständige Implementierung dieser Mechanismen.

Ein weiterer zentraler Aspekt betrifft die Sicherheit der Anwendung. In der manuellen Implementierung erfolgt die Verarbeitung von Benutzereingaben ausschließlich auf Basis von Zeichenkettenvergleichen. Die Eingaben werden normalisiert, beispielsweise durch Umwandlung in Kleinbuchstaben, und nicht als ausführbarer Code interpretiert. Dadurch wird das Risiko von Code-Injektionen reduziert.

Das Tabulator-Framework bietet zusätzlich integrierte Mechanismen zur sicheren Verarbeitung von Eingaben. Filterwerte werden intern verarbeitet und nicht direkt als ausführbarer Code interpretiert, wodurch ein weiterer Schutz vor typischen Angriffsszenarien wie Cross-Site-Scripting (XSS) gegeben ist.

Insgesamt wird deutlich, dass sowohl die funktionalen Anforderungen an die Datenverarbeitung als auch grundlegende Sicherheitsaspekte innerhalb der Anwendung berücksichtigt wurden.

4.6 Visualisierung und Interaktionslogik

Zur Ergänzung der tabellarischen Darstellung wurde eine grafische Visualisierung der Emissionsdaten integriert. Ziel ist es, Zusammenhänge schneller erfassbar zu machen und dem Nutzende sowohl aggregierte als auch detaillierte Analysen zu ermöglichen.

Die Erstellung der Diagramme erfolgt über eine zentrale Funktion, die je nach Kontext unterschiedliche Parameter erhält. Dabei wird vor der Neuerstellung eines Diagramms ein bestehendes Objekt entfernt, um Darstellungsfehler zu vermeiden:

```
function createChart(type, labels, values, labelText) {  
  if (emissionsChart) {  
    emissionsChart.destroy();  
  }  
  
  emissionsChart = new Chart(chartContext, {  
    type: type,  
    data: {  
      labels: labels,  
      datasets: [{  
        label: labelText,  
        data: values  
      }]  
    }  
  });  
}
```

Listing 7: Initialisierung und Neuerstellung des Diagramms mit Chart.js

Die Interaktionslogik basiert auf der dynamischen Anpassung der Visualisierung in Abhängigkeit von den aktuell sichtbaren Daten. Hierbei wird geprüft, ob bestimmte Filter aktiv sind, um zwischen unterschiedlichen Darstellungsformen zu unterscheiden (vgl. Anhang B, Listing B3).

Dieses Vorgehen ermöglicht eine kontextabhängige Darstellung der Daten. Während ohne aktive Filter eine aggregierte Übersicht nach Ländern dargestellt wird, erlaubt die gefilterte Ansicht eine detaillierte Betrachtung einzelner Unternehmen.

Die Verbindung zwischen Tabelle und Visualisierung wird über Events realisiert. Sobald sich die sichtbaren Daten ändern, wird das Diagramm automatisch aktualisiert. Dadurch wird sichergestellt, dass beide Komponenten stets denselben Datenstand widerspiegeln.

Insgesamt trägt die Visualisierung dazu bei, die Analyse der Daten zu unterstützen, ohne dabei die Komplexität der Anwendung unnötig zu erhöhen. Sie stellt somit eine sinnvolle Ergänzung zur tabellarischen Darstellung dar.

5. Schlussbetrachtung und Bewertung der Umsetzung

Im Rahmen der vorliegenden Fallstudie wurde mit „ATMOS Track“ eine webbasierte Anwendung zur Darstellung, Analyse und Visualisierung von CO₂-Emissionsdaten entwickelt. Ausgangspunkt war die Anforderung, für eine Non-Profit-Organisation eine öffentlich zugängliche Webseite bereitzustellen, die Transparenz über Emissionen von Unternehmen und Ländern schafft. Die entwickelte Lösung greift diese Zielsetzung auf, indem sie fiktive Emissionsdaten in einer strukturierten, filterbaren und visuell ergänzten Form bereitstellt.

Die zentralen Anforderungen der Aufgabenstellung wurden in der Anwendung umgesetzt. Die Webseite verfügt über einen eindeutigen Titel, ein Logo, eine globale Navigation, einen strukturierten Inhaltsbereich sowie einen Footer mit rechtlichen Hinweisen. Ergänzend wurde ein lokales Menü mit Ankerlinks integriert, das den Zugriff auf einzelne Seitenbereiche erleichtert. Durch den Einsatz von Bootstrap und responsiven Layoutklassen ist die Anwendung auf unterschiedlichen Bildschirmgrößen nutzbar. Damit werden sowohl Desktop-Ansichten als auch mobile Nutzungsszenarien berücksichtigt.

Ein Schwerpunkt der Umsetzung liegt in der tabellarischen Darstellung der CO₂-Emissionsdaten. Die Framework-basierte Lösung mit Tabulator erfüllt die Anforderungen an Sortier- und Filterfunktionen nach Land und Unternehmen auf effiziente Weise. Zusätzlich werden weitere Funktionen wie Pagination und visuelle Hervorhebungen genutzt, um die Übersichtlichkeit der Tabelle zu verbessern. Die ergänzend entwickelte manuelle Tabellenvariante verdeutlicht, wie dieselben Grundfunktionen ohne spezialisiertes Framework umgesetzt werden können. Dadurch wird nachvollziehbar, welche Verarbeitungsschritte bei der Nutzung eines Frameworks vereinfacht oder abstrahiert werden.

Die Verbindung der Tabelle mit einer grafischen Visualisierung stellt einen zusätzlichen Mehrwert dar. Die Diagramme ermöglichen es, Muster und Unterschiede in den Emissionsdaten schneller zu erfassen. Durch die dynamische Aktualisierung in Abhängigkeit von Sortier- und Filtervorgängen bleiben Tabelle und Visualisierung miteinander verbunden. Dadurch wird die Anwendung nicht nur zu einer reinen Datentabelle, sondern zu einem einfachen interaktiven Dashboard.

Auch grundlegende Sicherheitsaspekte wurden berücksichtigt. Benutzereingaben werden in der Anwendung nicht als ausführbarer Code interpretiert, sondern zur Filterung der vorhandenen Daten verwendet. Besonders in der manuellen Tabellenimplementierung wird durch die Verwendung von DOM-Methoden und `textContent` sichergestellt, dass ausgegebene Inhalte als Text behandelt werden. Dadurch wird das Risiko reduziert, dass manipulierte Inhalte als HTML oder JavaScript ausgeführt werden.

Insgesamt zeigt die Umsetzung, dass moderne Webtechnologien wie HTML, CSS und JavaScript in Verbindung mit Frameworks eine effiziente Entwicklung interaktiver Weboberflächen ermöglichen. Gleichzeitig verdeutlicht die manuelle Implementierung, dass ein grundlegendes Verständnis der zugrunde liegenden Mechanismen weiterhin notwendig ist, um Datenverarbeitung, DOM-Manipulation und Sicherheitsaspekte beurteilen zu können.

Für eine zukünftige Weiterentwicklung der Anwendung bieten sich mehrere Erweiterungen an. Dazu zählen insbesondere die Integration realer und regelmäßig aktualisierter Emissionsdaten, zusätzliche Filtermöglichkeiten, weitere Diagrammtypen sowie eine stärkere Ausrichtung auf Barrierefreiheit und Mehrsprachigkeit. Für einen produktiven Einsatz wäre außerdem eine serverseitige Datenquelle sinnvoll, über die Daten zentral gepflegt und validiert werden können. Die vorliegende Umsetzung bildet damit eine funktionale Grundlage, die die Anforderungen der Fallstudie erfüllt und zugleich sinnvolle Erweiterungspotenziale aufzeigt.

Literaturverzeichnis

Bootstrap (2024): *Bootstrap Documentation*. Abgerufen am 01.05.2026, von <https://getbootstrap.com/>

Chart.js (2024): *Chart.js Documentation*. Abgerufen am 01.05.2026, von <https://www.chartjs.org/>

European Environment Agency (2022): *Greenhouse gas emissions in Europe*. Abgerufen am 01.05.2026, von <https://www.eea.europa.eu/ims/greenhouse-gas-emissions-in-europe>

OpenAI. (2026). *ChatGPT [Large language model]*. Abgerufen am 01.05.2026, von <https://chatgpt.com/>

Tabulator (2024): *Tabulator: Interactive tables for the web*. Abgerufen am 01.05.2026, von <https://tabulator.info/>

Umweltbundesamt (2023): *Treibhausgas-Emissionen in Deutschland*. Abgerufen am 01.05.2026, von <https://www.umweltbundesamt.de/themen/klima-energie/treibhausgas-emissionen>

Anhang A: Technische Implementierung der Datenverarbeitung

```
function initApp(emissionsData) {  
    let currentData = [...emissionsData];  
}
```

Listing A1: Initialisierung der Anwendung

```
function renderTable(data) {  
    tableBody.textContent = "";  
  
    data.forEach(entry => {  
        const row = document.createElement("tr");  
  
        ["company", "country", "industry", "year", "emission",  
        "change"].forEach(field => {  
            const cell = document.createElement("td");  
            cell.textContent = entry[field];  
            row.appendChild(cell);  
        });  
  
        tableBody.appendChild(row);  
    });  
}
```

Listing A2: Rendern der Tabelle

```
function sortByCountry() {  
    currentData.sort((a, b) => a.country.localeCompare(b.country));  
    renderTable(currentData);  
}  
  
function sortByCompany() {  
    currentData.sort((a, b) => a.company.localeCompare(b.company));  
    renderTable(currentData);  
}  
  
function sortByIndustry() {  
    currentData.sort((a, b) => a.industry.localeCompare(b.industry));  
    renderTable(currentData);  
}
```

Listing A3: Sortierfunktionen

```

document.getElementById("sortCompany").onclick = () => {
    sortByCompany(currentData);
};

document.getElementById("sortCountry").onclick = () => {
    sortByCountry(currentData);
};

document.getElementById("sortIndustry").onclick = () => {
    sortByIndustry(currentData);
};

```

Listing A4: Event-Handler für Sortierung

```

searchInput.addEventListener("input", function () {
    const filterText = searchInput.value.toLowerCase().trim();

    currentData = emissionsData.filter(entry => {
        return entry.company.toLowerCase().includes(filterText) ||
               entry.country.toLowerCase().includes(filterText) ||
               entry.industry.toLowerCase().includes(filterText);
    });

    renderTable(currentData);
});

```

Listing A5: Such- und Filterfunktion

Anhang B: Implementierung der Benutzeroberfläche

```
<nav class="navbar navbar-expand-lg navbar-centered">
  <div class="container-fluid">

    <!-- Logo -->
    <a class="navbar-brand" href="home.html">
      
    </a>

    <!-- Hamburger -->
    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-
controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
      <span class="navbar-toggler-icon"></span>
    </button>

    <!-- Globale Navigation -->
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">

        <div class="dropdown">
          <button class="btn btn-outline-secondary
dropdown-toggle" type="button" data-bs-
toggle="dropdown" aria-expanded="false">
            Auf dieser Seite
          </button>
          <ul class="dropdown-menu">
            <li>
              <a class="dropdown-item" h
ref="#uebersicht">Übersicht</a>
            </li>
            <li>
              <a class="dropdown-item"
href="#fallstudie">Fallstudie</a>
            </li>
            <li>
              <a class="dropdown-item"
href="#services">Services</a>
            </li>
          </ul>
        </div>

        <li class="nav-item">
          <a class="nav-link active" aria-current="page"
href="home.html">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="table.html">Tabelle</a>
        </li>
        <li class="nav-item">
          <a class="nav-link"
href="vanilla.html">Vanilla</a>
        </li>
        <li class="nav-item">
          <a class="nav-link"
href="frameworks.html">Frameworks</a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

```

        </li>
      </ul>
    </div>
  </div>
</nav>

```

Listing B1: Navigationsleiste

```

{
  title: "Veränderung (%)",
  field: "change",
  sorter: "number",
  minWidth: 160,
  formatter: function(cell) {
    let value = cell.getValue();

    if (value > 0) {
      return `

```

Die Formatter-Funktion prüft den jeweiligen Wert der Spalte „Veränderung (%)“. Positive Werte werden rot und fett dargestellt, während negative oder neutrale Werte grün ausgegeben werden. Dadurch werden steigende und sinkende Emissionswerte visuell schneller erfassbar.

Listing B2: Farbliche Hervorhebung von Emissionsveränderungen

```

function updateChart(visibleData) {
  const filters = table.getHeaderFilters();

  const hasCountryFilter = filters.some(f => f.field === "country");
  const hasIndustryFilter = filters.some(f => f.field === "industry");

  if (hasCountryFilter || hasIndustryFilter) {
    createChart("bar",
      visibleData.map(entry => entry.company),
      visibleData.map(entry => entry.emission),
      "Emissionen in t CO2"
    );
  } else {
    const grouped = groupEmissionByField(visibleData, "country");

    createChart("pie",
      Object.keys(grouped),
      Object.values(grouped),
      "Emissionen nach Land"
    );
  }
}

```

Listing B3: Aktualisierung des Diagramms auf Basis der Tabelleninteraktion